

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\interpolation\LinearInterpolator.java

- Specification

module LinearInterpolator

type

x, y: sequence of real

n: int

ex: set of {

None,

NullPointerException,

NoDataException,

DimensionMismatchException,

NumbersTooSmallException,

NonMonotonicSequenceException,

NotFiniteNumberException

}

process Interpolate(x, y: sequence of real) outputs: ex: set of {...}

Functional Scenario Form

x = 0

&& ex = NullPointerException ||

|x| = 0

&& ex = NoDataException ||

|x| > |y|

&& ex = DimensionMismatchException ||

|x| < 2

&& ex = NumbersTooSmallException ||

finite = 0

&& ex = NotFiniteNumberException ||

x ≤ i-1

&& ex = NonMonotonicSequenceException ||

finite = 1

&& ex = None

end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.exception;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
/**
 * Exception to be thrown when the a sequence of values is not monotoni
cally
 * increasing or decreasing.
 *
 * @since 2.2 (name changed to "NonMonotonicSequenceException" in 3.0)
 */
public class NonMonotonicSequenceException extends MathIllegalNumberExc
eption {
    /** Serializable version Id. */
    private static final long serialVersionUID = 20210531L;
    /**
     * Whether the sequence should be increasing.
     */
    private final boolean increasing;
    /**
     * Whether the sequence must be strictly increasing or decreasing.
     */
    private final boolean strict;
```

```

/**
 * Index of the wrong value.
 */
private final int index;
/**
 * Previous value.
 */
private final Number previous;
/**
 * Construct the exception.
 * This constructor uses default values assuming that the sequence
should
 * have been strictly increasing.
 *
 * @param wrong Value that did not match the requirements.
 * @param previous Previous value in the sequence.
 * @param index Index of the value that did not match the requireme
nts.
 */
public NonMonotonicSequenceException(Number wrong,
                                     Number previous,
                                     int index) {
    this(wrong, previous, index, true, true);
}
/**
 * Construct the exception.
 *
 * @param wrong Value that did not match the requirements.
 * @param previous Previous value in the sequence.
 * @param index Index of the value that did not match the requireme
nts.
 * @param increasing {@code true} for a sequence required to be
 * increasing, {@code false} for a decreasing sequence.
 * @param strict Whether the sequence must be strictly increasing o
r
 * decreasing.
 */
public NonMonotonicSequenceException(Number wrong,
                                     Number previous,
                                     int index,
                                     boolean increasing,
                                     boolean strict) {
    super(increasing ?
          (strict ?

```

```

        LocalizedFormats.NOT_STRICTLY_INCREASING_SEQUENCE :
        LocalizedFormats.NOT_INCREASING_SEQUENCE) :
        (strict ?
        LocalizedFormats.NOT_STRICTLY DECREASING_SEQUENCE :
        LocalizedFormats.NOT DECREASING_SEQUENCE),
        wrong, previous, Integer.valueOf(index), Integer.valueOf(
index - 1));
        this.increasing = increasing;
        this.strict = strict;
        this.index = index;
        this.previous = previous;
    }
    /**
     * @return {@code true} if the sequence should be increasing.
     */
    public boolean getIncreasing() {
        return increasing;
    }
    /**
     * @return {@code true} is the sequence should be strictly monotoni
c.
     */
    public boolean getStrict() {
        return strict;
    }
    /**
     * Get the index of the wrong value.
     *
     * @return the current index.
     */
    public int getIndex() {
        return index;
    }
    /**
     * @return the previous value.
     */
    public Number getPrevious() {
        return previous;
    }
}

```

- Coverage report

1. Test case number: 10

2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LinearInterpolator.java	8.5 %	10	108	118
LI_VT	5.6 %	2	34	36
LinearInterpolator	100.0 %	8	0	8
interpolate(double[], double[])	100.0 %	8	0	8
LI_Runner		0	0	0

3. V-Method testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LinearInterpolator.java	14.4 %	304	1,809	2,113
LI_VT	34.3 %	191	366	557
LI_Runner	62.5 %	5	3	8
LinearInterpolator	100.0 %	108	0	108
interpolate(double[], double[])	100.0 %	105	0	105

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LinearInterpolator.java	19.5 %	23	95	118
LI_FT	40.5 %	17	25	42
LinearInterpolator	75.0 %	6	2	8
interpolate(double[], double[])	75.0 %	6	2	8

5. Fuzz testing instruction coverage: 82.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LinearInterpolator.java	31.1 %	658	1,455	2,113
LI_FT	66.2 %	569	291	860
LinearInterpolator	82.4 %	89	19	108
interpolate(double[], double[])	81.9 %	86	19	105

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LinearInterpolator.java	16.9 %	20	98	118
LI_Z3	37.5 %	12	20	32
LinearInterpolator	100.0 %	8	0	8
interpolate(double[], double[])	100.0 %	8	0	8
LI_Runner		0	0	0

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LinearInterpolator.java	25.0 %	528	1,585	2,113
LI_Runner	62.5 %	5	3	8
LI_Z3	71.6 %	415	165	580
LinearInterpolator	100.0 %	108	0	108
interpolate(double[], double[])	100.0 %	105	0	105